

NumPy 1 – Introduction

1. What is NumPy?

NumPy = Numerical Python

- Used for numerical calculations, arrays & matrices.
- Core of scientific computing, Machine Learning & Deep Learning.

Main Data Structure:

NumPy array (ndarray)

Python + Fast numerical computation + Arrays

-- Concept Map --

[Python code] → NumPy Array ('ndarray') → [Fast ML Ops]

2. Why is NumPy Important?

ML math heavily involves Vectors, Matrices, Dot products, Matrix multiplication, & Gradients.

Example (Linear Regression):

$\text{prediction} = Xw + b$ (where X and w are NumPy arrays)

3 & 4. Python Lists vs NumPy Arrays & Vectorization

Python lists are general-purpose containers with high memory overhead. NumPy arrays are optimized for numbers.

Python List

`numbers = [1, 2, 3, 4]`

Slow loops, dynamic types, high memory

NumPy Array

`numbers = np.array([1, 2, 3, 4])`

C-optimized, contiguous memory, fast

Vectorization Definition:

Performing operations on an entire array without writing explicit Python loops.

Instead of: `for x in arr: result.append(x * 2)` → Simply do: `arr * 2`

5. N-Dimensional Arrays

1D → *Vector* e.g., [1, 2, 3] / Shape: (3,)

2D → *Matrix* e.g., [[1, 2], [3, 4]] / Shape: (2, 2)

3D → *Collection of matrices* (Used in Images, Videos, Deep Learning)

Hierarchy: 1D → 2D → 3D → ... → N-D

6. NumPy in Machine Learning

Essential when implementing ML algorithms from scratch:

- *Linear Regression*: $\text{prediction} = Xw + b$
- *Gradient Descent*: $w = w - \text{learning_rate} \times \text{gradient}$
- *Neural Networks*: Weights, biases, matrix multiplication, activation functions, backpropagation.

Helps understand and implement math behind ML!

7. NumPy vs Pandas

Feature	NumPy	Pandas
Domain	Numerical computing	Data analysis
Core Structure	Arrays ('ndarray')	Series & DataFrames
Use Case	Fast math & ML math	Data cleaning & tabular manipulation

Workflow: Dataset → Pandas (Clean/Prepare) → NumPy (Numerical Ops) → ML Model

8 & 9. Memory Efficiency & Contiguous Storage

Python List Memory

Stores references to objects → extra overhead & memory.

NumPy Array Memory

Stores numerical values in a compact block with fixed dtype ('int32', 'float64').

Contiguous Memory Block:

[1][2][3][4][5][6]

↑ Continuous storage helps CPU access data efficiently!

10. Broadcasting

Allows operations between arrays of compatible shapes without manually repeating values or loops.

```
arr = np.array([1, 2, 3])
arr + 10 → [11, 12, 13]
```

11. Important NumPy Concepts to Learn

1. Basics

`np.array()`, `shape`, `ndim`, `size`, `dtype`

2. Array Creation

`zeros()`, `ones()`, `arange()`, `linspace()`, `random`

3. Operations

Indexing, Slicing, Reshaping, Transpose, Concatenation

4. Key Concepts

Vectorization, Broadcasting, Axis, Views vs copies

12. Big Picture & One-Line Summary

One-Line Summary:

NumPy is a fast, memory-efficient library for numerical data, arrays, vectors, and matrices—fundamental for ML & scientific computing!